ADVANCED AI ENGINEERING RESEARCH MANUAL

Large Language Model Fine-Tuning, LoRA/QLoRA, Quantization, GPU Mathematics & Graph RAG

Author: Sardar Ahmed | Alphatron Technologies Research Internship (Batch 2 - Week 4)

Mandatory Research Manual (Task 3 & 4) | August 2026

TABLE OF CONTENTS

---

1. EXECUTIVE OVERVIEW & FOUNDATIONAL PARADIGMS

Modern AI engineering balances two foundational pillars: Domain Specialization (adapting general foundation models to proprietary knowledge, styles, and task distributions) and Computational Efficiency (training and serving multi-billion parameter architectures under tight GPU memory and latency budgets).

This technical manual provides a comprehensive, mathematically rigorous, and architecturally exhaustive deep dive into model adaptation, parameter-efficient fine-tuning (PEFT), quantization, distributed GPU sizing, and retrieval paradigms (Naive vs. Graph RAG).

---

2. SECTION 1: TRANSFER LEARNING IN DEEP LEARNING & NLP

1.1 Conceptual Foundations & Mathematical Formulation

Transfer Learning is a machine learning paradigm where knowledge acquired from solving a source task in a source domain is leveraged to enhance generalization on a target task in a target domain.

Formally, a domain D = {X, P(X)} consists of a feature space X and a marginal probability distribution P(X). A task T = {Y, P(Y|X)} consists of a label space Y and a conditional objective distribution P(Y|X).

Given a source domain D_S with task T_S, and target domain D_T with task T_T, Transfer Learning optimizes P(Y_T|X_T) using knowledge from D_S and T_S, where D_S != D_T or T_S != T_T.

1.2 Transfer Learning Taxonomies

1. Inductive Transfer Learning: Target task differs from source task (T_T != T_S). Target domain contains labeled data used to update model representations.

2. Transductive Transfer Learning: Source and target tasks are identical (T_S = T_T), but source and target domains differ (D_S != D_T). Often leveraged in domain adaptation across languages or text registers.

3. Unsupervised Transfer Learning: Both tasks differ and lack explicit supervised labels, focusing on representation learning across distinct modalities.

---

3. SECTION 2: LARGE LANGUAGE MODEL (LLM) FINE-TUNING PARADIGMS

2.1 The Training Continuum: Pre-training to Post-Training Alignment

The lifecycle of modern Large Language Models consists of three distinct phases:

- Pre-Training: Self-supervised next-token prediction across trillions of tokens on massive web-scale corpora.
- Supervised Fine-Tuning (SFT): Instruction-tuning using high-quality prompt-response pairs to instill conversational assistant capabilities.
- Preference Alignment (DPO / PPO / GRPO): Optimizing responses based on human/AI preference feedback, safety constraints, and chain-of-thought reasoning.

2.2 Supervised Fine-Tuning (SFT) Mathematics

Given a dataset D_SFT = {(x_i, y_i)}, model parameters theta are trained by minimizing autoregressive cross-entropy loss over target tokens:

\\mathcal{L}_{SFT}(\\theta) = - \\sum_{i=1}^N \\sum_{t=1}^{|y^{(i)}|} \\log P_\\theta\\left(y_t^{(i)} \\mid x^{(i)}, y_{<t}^{(i)}\\right)

Prompt tokens x_i are masked with label -100 so that gradients are computed exclusively on the completion tokens y_i.

2.3 Post-SFT Alignment: DPO & GRPO

1. Direct Preference Optimization (DPO): Optimizes policy directly using pairwise preference loss without training an explicit reward model.

2. Group Relative Policy Optimization (GRPO): Samples a group of outputs for each prompt, standardizes rewards across the group, and updates the policy without training a separate critic network.

---

4. SECTION 3: DATASET PREPARATION, FORMATTING & ALIGNMENT

3.1 Standard Industry Formats

- Alpaca Format: Single-turn schema with instruction, input, and output fields.
- ShareGPT / ChatML Format: Multi-turn schema with role-tagged messages (system, human, gpt).

3.2 Tokenization & Loss Masking Pipeline

1. Apply model-specific chat template (e.g. ChatML, Llama-3, Gemma).

2. Tokenize using Byte-Pair Encoding (BPE) or SentencePiece.

3. Mask user prompt tokens with label -100.

4. Dynamic right-padding to max sequence length with packed batching.

---

5. SECTION 4: LOW-RANK ADAPTATION (LoRA)

4.1 Intrinsic Dimension & LoRA Derivation

Aghajanyan et al. (2020) proved that over-parameterized neural networks have a low intrinsic dimension during downstream adaptation. LoRA decomposes weight updates Delta W into two low-rank factor matrices:

\\Delta W = \\frac{\\alpha}{r} (B \\cdot A)

Where W_0 in R^(d x k), B in R^(d x r), A in R^(r x k), and rank r << min(d, k) (typically r in {8, 16, 32, 64}).

- Matrix A is initialized from a Gaussian distribution: N(0, sigma^2).
- Matrix B is initialized to zeros (B = 0), ensuring Delta W = 0 at the start of training.
- Setting alpha = 2r provides stable gradient scaling across different rank choices.

6. SECTION 5: QUANTIZED LOW-RANK ADAPTATION (QLoRA)

5.1 The 3 Foundational Innovations of QLoRA

1. 4-bit NormalFloat (NF4): Information-theoretically optimal quantile quantization for normally distributed neural weights.

2. Double Quantization (DQ): Quantizes quantization constants to save 0.37 bits per parameter.

3. Paged Optimizers: Leverages CUDA Unified Memory to page optimizer states to CPU RAM during memory spikes to prevent out-of-memory (OOM) crashes.

---

7. SECTION 6: MODEL QUANTIZATION TECHNOLOGIES

7.1 PTQ vs. QAT

- Post-Training Quantization (PTQ): Compresses weights after pre-training using small calibration datasets without updating model parameters. Ultra-fast, zero backprop overhead.
- Quantization-Aware Training (QAT): Simulates low-precision rounding errors during training using Straight-Through Estimators (STE). Achieves superior perplexity preservation at extreme bit-widths (e.g. INT2/INT3).

7.2 Modern Quantization Formats Comparison Matrix

Format	Bit Width	Target Hardware	Primary Runtime	Key Advantage
FP16 / BF16	16-bit	High-End GPUs (A100/H100)	Native PyTorch	Baseline gold standard accuracy
FP8 (E4M3/E5M2)	8-bit	NVIDIA Ada / Hopper (H100/L40)	TensorRT-LLM / vLLM	2x speedup with zero perplexity loss
BitsAndBytes NF4	4-bit	Consumer & Cloud GPUs	HuggingFace PEFT	Seamless fine-tuning & training
AWQ	4-bit	NVIDIA GPUs / vLLM	AutoAWQ / vLLM	Protects top 1% salient weight channels
GPTQ	4-bit / 8-bit	NVIDIA GPUs / ExLlamaV2	AutoGPTQ / ExLlamaV2	Fast matrix multiplication on GPUs
GGUF	2-bit to 8-bit	CPU + Apple Silicon + GPU	llama.cpp / Ollama	Universal edge & multi-thread CPU inference

---

8. SECTION 7: GPU MEMORY MATHEMATICS & HARDWARE SIZING

8.1 Exact Memory Equation for Training

Total GPU VRAM required during training is governed by:

\\text{VRAM}_{Train} = M_{weights} + M_{gradients} + M_{optimizer} + M_{activations} + M_{KV-cache} + M_{CUDA-overhead}

For a model with P billion parameters:

1. Model Weights (M_weights): FP16/BF16: P 2 GB | QLoRA (NF4): P 0.5 GB

2. Gradients (M_gradients): Full Fine-Tuning: P 2 GB | LoRA: Negligible (< 0.05 GB)

3. Optimizer States (M_optimizer) for AdamW: Full Fine-Tuning: P 12 GB to P 16 GB | LoRA/QLoRA: < 0.1 GB

4. Activations (M_activations): With Gradient Checkpointing: M_act ~ B S L D 2

8.2 Exact Memory Equation for Inference

\\text{VRAM}_{Inference} = M_{weights} + M_{KV-Cache} + M_{Context-Activations}

\\text{KV-Cache Memory} = 2 \\times 2 \\text{ (bytes/FP16)} \\times L \\times N_{KV} \\times D_{head} \\times S_{context} \\times B

---

9. SECTION 8: GPU MEMORY UTILIZATION & OPTIMIZATION TECHNIQUES

1. FlashAttention-2 & 3: IO-aware tiling in GPU SRAM reducing memory complexity from O(N^2) to O(N), speeding up attention by 3-5x.

2. Gradient Checkpointing: Discards forward activations and recomputes them during backward pass, saving up to 70% activation VRAM.

3. ZeRO (Zero Redundancy Optimizer):

- ZeRO-1: Shards optimizer states across data-parallel ranks (4x memory reduction).
- ZeRO-2: Shards optimizer states + gradients (8x memory reduction).
- ZeRO-3 / FSDP: Shards optimizer states + gradients + model parameters (enables training 70B+ models across multi-GPU nodes).

---

10. SECTION 9: COMPARATIVE ANALYSIS: LLM FINE-TUNING VS. RAG WITH VECTOR DATABASES

10.1 Parametric vs. Non-Parametric Memory

- Fine-Tuning modifies Parametric Memory: Encodes style, domain terminology, syntactic rules, and specialized reasoning pathways directly into neural network weights.
- RAG modifies Non-Parametric Memory: Dynamically retrieves factual context from an external vector index or graph and injects it into the prompt at inference time.

10.2 Multi-Dimensional Comparison Matrix

| Evaluation Dimension | Large Language Model Fine-Tuning | Retrieval-Augmented Generation (RAG) |

| :--- | :--- | :--- |

| Knowledge Freshness | Static (requires periodic retraining) | Real-Time (instant index update) |

| Hallucination Control | Moderate (model can still fabricate) | High (grounded in retrieved context) |

| Exact Source Attribution | None (weights cannot provide URLs/pages)| 100% Verifiable Source Citations |

| Behavior / Style Alignment| Exceptional (adopts tone, syntax, schema)| Weak (guided only via system prompt) |

| Domain Jargon Mastery | High (updates token embeddings) | Dependent on chunk context |

| Computational Upfront Cost| High (GPU clusters, dataset curation) | Low (Vector store + embedding model) |

| Inference Latency | Fast (standard model forward pass) | Additional vector search + larger prompt |

| Data Privacy & Access Control| Difficult to enforce role-based access | Strict Document-Level ACLs |

---

11. SECTION 10: COMPARATIVE ANALYSIS: NAIVE RAG VS. ADVANCED GRAPH RAG

11.1 The Limitations of Naive Vector RAG

Naive RAG splits documents into arbitrary chunks and computes semantic embeddings. While effective for localized point queries, it struggles with:

1. Multi-Hop Reasoning Blindspot: Cannot traverse relational links between disparate entities across separated document chunks.

2. Holistic / Global Synthesis Failure: Incapable of answering corpus-wide global questions.

3. Context Fragmentation: Semantic chunking breaks relationships spanning chunk boundaries.

11.2 Architectural Comparison Matrix

| Architectural Dimension | Naive Vector RAG | Advanced Graph RAG |

| :--- | :--- | :--- |

| Data Structure | Flat vector embeddings in metric space | Structured Entity-Relation Graph + Vector Index |

| Search Mechanism | k-Nearest Neighbor (k-NN) / Cosine Similarity | Graph Traversal (PageRank, Cypher) + Community Search |

| Multi-Hop Traversal | Fails on > 2 hops | Traverses multi-degree entity graphs |

| Global Corpus Summaries | Impossible (cannot fit all chunks in prompt)| Hierarchical Community Summarization |

| Indexing Complexity | Low (O(N) chunk embeddings) | High (LLM-based entity/relation extraction) |

| Indexing Cost | Fast & Inexpensive (< .01 per document) | Requires LLM passes per chunk (5-20x cost) |

| Hallucination Resilience | Moderate | Ultra-High (Explicit semantic relationships) |

---

12. SECTION 11: ENTERPRISE DECISION MATRIX, TRADE-OFFS & HYBRID IMPLEMENTATION FRAMEWORK

The optimal enterprise architecture combines Fine-Tuned Domain Specialist Models (for specialized jargon, syntax, and formatting) with Graph RAG Context Ingestion (for real-time freshness, multi-hop truth verification, and strict ACL compliance).

12.1 Practical PyTorch & HuggingFace QLoRA Snippet

`python

import torch

from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig

from peft import LoraConfig, get_peft_model, prepare_model_for_kbit_training

1. Configure 4-bit NF4 Quantization

bnb_config = BitsAndBytesConfig(

load_in_4bit=True,

bnb_4bit_quant_type='nf4',

bnb_4bit_use_double_quant=True,

bnb_4bit_compute_dtype=torch.bfloat16,

)

2. Load Base Foundation Model

model_id = 'meta-llama/Meta-Llama-3-8B'

tokenizer = AutoTokenizer.from_pretrained(model_id)

model = AutoModelForCausalLM.from_pretrained(model_id, quantization_config=bnb_config, device_map='auto')

3. Prepare Model & Inject LoRA Adapters

model = prepare_model_for_kbit_training(model)

lora_config = LoraConfig(

r=16,

lora_alpha=32,

target_modules=['q_proj', 'k_proj', 'v_proj', 'o_proj', 'gate_proj', 'up_proj', 'down_proj'],

lora_dropout=0.05,

bias='none',

task_type='CAUSAL_LM',

)

peft_model = get_peft_model(model, lora_config)

peft_model.print_trainable_parameters()

`

---

13. ACADEMIC & INDUSTRY REFERENCES

1. Hu, E. J., et al. (2021). LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685.

2. Dettmers, T., et al. (2023). QLoRA: Efficient Finetuning of Quantized LLMs. NeurIPS 2023. arXiv:2305.14314.

3. Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020.

4. Edge, D., et al. (2024). From Local to Global: A Graph RAG Approach to Query-Focused Summarization. Microsoft Research. arXiv:2404.16130.

5. Dao, T., et al. (2022). FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. NeurIPS 2022.

6. Rafailov, R., et al. (2023). Direct Preference Optimization: Your Language Model is Secretly a Reward Model. NeurIPS 2023.

7. Rajbhandari, S., et al. (2020). ZeRO: Memory Optimizations Toward Training Trillion Parameter Models. IEEE/ACM SC20.

8. Frantar, E., et al. (2022). GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. arXiv:2210.17323.

9. Lin, J., et al. (2023). AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration. MLSys 2024.

10. Aghajanyan, A., et al. (2020). Intrinsic Dimensionality Explains the Effectiveness of Language Model Fine-Tuning. ACL 2021.